

GUIA_EJECUCION_PRUEBAS

Guía de ejecución de pruebas y reportes de cobertura

Equipo: Hawk Durant, Emilio Jaramillo, Rodrigo Candia

Proyecto: Backend S.I.G.I

1. Requisitos previos

- **JDK 17** (`java -version`)
- **Maven 3.9+** (`mvn -version`)
- Conexión a internet la primera vez (descarga de dependencias Maven)

No hace falta Docker ni MySQL para las pruebas unitarias.

2. Ejecutar todas las pruebas

Desde la raíz del repositorio `sigi-backend` :

```
mvn clean verify
```

Esto en cada módulo:

1. Compila el código
2. Ejecuta los tests (JUnit 5)
3. Genera reporte JaCoCo HTML
4. Valida que cada servicio de negocio tenga ≥ 70 % de cobertura de líneas

Duración aproximada: 10–15 minutos en un MacBook estándar (incluye tests `@WebMvcTest` que levantan contexto Spring parcial).

3. Ejecutar un solo microservicio

```
cd servicio-usuario  
mvn clean verify
```

O desde la raíz sin cambiar de carpeta:

```
mvn verify -pl servicio-usuario
```

Reanudar el reactor si falló un módulo intermedio:

```
mvn verify -rf :servicio-reporte
```

4. Solo tests (sin chequeo de cobertura)

```
mvn test
```

Útil durante desarrollo cuando todavía no llegamos al 70 %.

5. Ver reportes de cobertura

Tras `mvn verify`, abrir en el navegador:

Reporte	Ruta
servicio-usuario	<code>servicio-usuario/target/site/jacoco/index.html</code>
servicio-reporte	<code>servicio-reporte/target/site/jacoco/index.html</code>
... (idem cada módulo)	<code>servicio-*/target/site/jacoco/index.html</code>
Consolidado	<code>target/site/jacoco-aggregate/index.html</code>

En macOS:

```
open servicio-usuario/target/site/jacoco/index.html
open target/site/jacoco-aggregate/index.html
```

6. Copiar reportes a docs/cobertura (entrega)

```
./docs/scripts/copiar-reportes-cobertura.sh
```

(Este script copia los HTML principales a `docs/cobertura/` para subirlos al repo de documentación.)

7. Estructura de archivos de prueba

```
pom.xml                ← JaCoCo y umbral 70 % (padre)
servicio-*/
  pom.xml              ← referencia al plugin JaCoCo
  src/test/java/...    ← clases *Test.java
  target/site/jacoco/index.html ← reporte generado (no commitear)
```

No **commitear** carpetas `target/`; están en `.gitignore`. Solo **commiteamos** el **código fuente** de las pruebas.

8. Swagger (alternativa a Postman)

Con Docker levantado, Swagger UI por servicio:

- servicio-usuario: `http://localhost:8081/swagger-ui.html`
- servicio-reporte: `http://localhost:8082/swagger-ui.html`
- (puertos 8083–8088 para el resto)

Detrás del Gateway usamos la colección Postman en `docs/postman/SIGI-Backend.postman_collection.json`.

9. Solución de problemas frecuentes

Problema	Causa probable	Qué hicimos nosotros
Unknown lifecycle phase "--coverage"	Sintaxis de npm, no de Maven	Usar <code>mvn verify</code> , no <code>mvn test -- --coverage</code>
Falla JaCoCo check	Cobertura bajo 70 %	Agregar tests en <code>service/</code> y <code>controller/</code>
<code>@WebMvcTest</code> lento o falla Eureka	Spring Cloud intenta registrarse	<code>@TestPropertySource</code> desactivando discovery en tests
Tests de media fallan al escribir disco	Permisos sandbox	Usar <code>@TempDir</code> en <code>MediaServiceTest</code>

10. Referencias

- Informe completo: [INFORME PRUEBAS UNITARIAS.md](#)
- Arquitectura: [ARQUITECTURA MICROSERVICIOS.md](#)
- Repositorios GitHub: [repositorios.txt](#)